AUDITFLOW: ENVIRONMENTAL AUDIT, INSPECTION, CAPA AND STATUTORY REPORTING PLATFORM

A PROJECT REPORT

Submitted by

MONISH H (310624148060)

in partial fulfillment for the award of the Degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (AI & ML)

2311CSE503R - FULL STACK DEVELOPMENT



EASWARI ENGINEERING COLLEGE

(AN AUTONOMOUS INSTITUTION) – AFFILIATED TO ANNA UNIVERSITY, CHENNAI

OCTOBER 2026

EASWARI ENGG COLLEGE

EASWARI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this mini project report titled **"AUDITFLOW: ENVIRONMENTAL AUDIT, INSPECTION, CAPA AND STATUTORY REPORTING PLATFORM"** is the Bonafide work of **"MONISH H (310624148060)"**, who carried out the project work under my supervision.

HEAD OF THE DEPARTMENT	**SUPERVISOR / PROJECT GUIDE**
Dr. Y. Justin Dhas	Ms. Divyanjali
Department of CSE (AI & ML)	Department of CSE (AI & ML)
Easwari Engineering College	Easwari Engineering College



EASWARI ENGINEERING COLLEGE

Certificate of Viva-voce-Examination

This is to certify that the Viva Voce Examination of the Project Work entitled **"AUDITFLOW: ENVIRONMENTAL AUDIT, INSPECTION, CAPA AND STATUTORY REPORTING PLATFORM"** submitted by **Mr. MONISH H (Reg. No. 310624148060)** in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering was conducted on _____ (Date) in the Department of Computer Science and Engineering (AI & ML), Easwari Engineering College, Ramapuram – Chennai.

Date: _____

Internal Examiner

Name : _____

Designation : _____

Address : _____

External Examiner

Name : _____

Designation : _____

Address : _____

ABSTRACT

Industrial environmental sustainability, regulatory compliance, and pollution abatement have become vital mandates across modern manufacturing, chemical processing, dyeing, and engineering facilities. Regulatory bodies such as the State Pollution Control Boards (e.g., TNPCB), the Central Pollution Control Board (CPCB), and international environmental bodies enforce strict statutory compliance standards under the Water (Prevention and Control of Pollution) Act of 1974 and the Air (Prevention and Control of Pollution) Act of 1981. However, conventional operational tracking remains crippled by paper-based clipboards, isolated desktop spreadsheets, uncoordinated email trails, and fragmented supervisory workstations. These antiquated practices introduce significant human error, lack audit provenance, fail to flag discharge limit exceedances in real-time, and make statutory return preparation tedious and delayed.

This project presents **AuditFlow**, an enterprise-grade, end-to-end full-stack compliance management platform engineered on the MERN technology stack (MongoDB, Express.js, React.js, and Node.js). AuditFlow delivers a unified ecosystem that transforms the entire compliance lifecycle: *"Audit it. Fix it. Report it."* The platform features dynamic, checklist-driven auditing across major environmental focal areas (water conservation, trade effluent treatment, ambient air emissions, hazardous waste storage, and consent conditions). When an audit is completed, the system automatically locks the record, computes section and overall compliance scores, and **auto-generates time-bound Corrective and Preventive Actions (CAPA)** for every non-compliance discovered, enforcing strict 7, 30, or 60-day remediation SLAs calibrated to finding severity.

Furthermore, AuditFlow introduces automated environmental inspection telemetry monitoring. Pre-save Mongoose validation hooks continuously evaluate discharge readings (e.g., BOD, COD, TSS, TDS, and PM2.5) against statutory consent limits; any exceedance immediately raises an automated CAPA incident. Corrective workflows follow a strict four-stage verification pipeline (Open ➔ In Progress ➔ Pending Verification ➔ Closed) with Role-Based Access Control (RBAC), ensuring that while facility managers record root-causes and upload Multer-backed photographic evidence, only authorized auditors or administrators can verify and close actions. An automated background cron daemon performs daily midnight scans, dynamically flagging overdue actions on an executive 60-day visual timeline ribbon. Finally, a statutory reporting engine synthesizes live operational data into regulatory returns with instant CSV export.

Keywords: Environmental Audits, MERN Stack, Automated CAPA, Statutory Reporting, Pollution Control, Limit Exceedance, Role-Based Access Control, Mongoose Hooks, Cloud Deployment.

ACKNOWLEDGEMENT

We express our sincere gratitude to our esteemed Principal **Dr. P. Deiva Sundari** and the Management of **Easwari Engineering College** for providing us with state-of-the-art computational infrastructure and an intellectually stimulating environment to carry out this project work successfully.

We convey our heartfelt thanks to the Head of the Department **Dr. Y. Justin Dhas**, Department of Computer Science and Engineering (AI & ML), for his continuous encouragement, academic support, and constructive feedback throughout the course of this full-stack development curriculum.

We are extremely thankful and indebted to our Project Guide **Ms. Divyanjali**, Department of Computer Science and Engineering (AI & ML), for her invaluable guidance, rigorous technical suggestions, and motivation throughout the conception, architecture, implementation, and deployment phases of this project.

We would also like to extend our warm appreciation and gratitude to all the esteemed faculty members and technical staff of the Department of Computer Science and Engineering (AI & ML) for their continuous support and cooperation.

Finally, I thank my beloved parents and friends for their enduring patience, encouragement, and moral support during the realization and completion of this project work.

MONISH H
(Reg. No. 310624148060)

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**4**
	ACKNOWLEDGEMENT	**5**
	LIST OF FIGURES	**8**
	LIST OF TABLES	**9**
1	**INTRODUCTION**	**10**
1.1 Overview		10
1.1.1 Industrial Chemistry & Pollution Abatement Fundamentals		10
1.1.2 National Environmental Policy Framework & TNPCB Mandates		11
1.2 Problem Definition		11
1.3 Objective		12
1.4 Scope of the Project		12
1.4.1 Simulation Boundary and Academic Clarification		12
1.5 Motivation		13
2	**LITERATURE SURVEY**	**14**
2.1 Literature Review		14
2.1.1 Summary of Surveyed Methodologies		15
2.2 Existing Systems		16
2.3 Proposed System		16
3	**SYSTEM ANALYSIS**	**18**
3.1 Problem Statement		18
3.2 Existing System Analysis		18
3.3 Proposed System Capabilities		18
3.4 Functional Requirements Specification		19
3.5 Non-Functional Requirements Specification		19
3.6 Feasibility Study		19
3.7 Industrial Effluent Stoichiometry & Discharge Mass Balance		20
3.7.1 BOD/COD Degradation Kinetics		20
3.7.2 Stack Gas Dispersion and Particulate Dynamics		21
3.8 Regulatory Framework and Industry Standards		21
4	**SYSTEM DESIGN**	**23**
4.1 System Architecture		23
4.2 Module Description		24
4.3 Use Case Diagram		24
4.4 Entity-Relationship (ER) Diagram		25
4.5 UI Navigation and Workflow Sequence		26
4.6 Database Design		26
4.7 API Design		27
4.8 Mathematical Compliance Model and Parameter Mapping		28
4.9 RESTful Data Transfer Protocols and Contract Specifications		29

CHAPTER NO.	TITLE	PAGE NO.
5	**SYSTEM IMPLEMENTATION**	**30**
5.1 Implementation Overview		30
5.2 Frontend Implementation		30
5.3 Backend Implementation		30
5.4 MongoDB Database Implementation		31
5.5 Authentication and Role-Based Access Control		31
5.6 Mathematical Calculations & Scoring Engine		31
5.6.1 Audit Scoring Formula		31
5.6.2 Dynamic SLA Deadline Assignment		31
5.6.3 Pre-Save Limit Exceedance Hooks		32
5.6.4 Daily Cron Scheduler Daemon		32
5.6.5 Network Interceptors and Session Hydration		32
5.6.6 Defensive Request Validation and Schema Sanitization		32
5.7 Deployment Architecture		33
6	**RESULTS AND DISCUSSION**	**34**
6.1 Testing Methodology		34
6.2 Functional Test Cases		34
6.3 Performance and Stress Testing Benchmarks		36
6.4 Defensive and Security Verification		36
6.5 User Acceptance Testing (UAT) and SUS Evaluation		36
7	**CONCLUSION AND FUTURE SCOPE**	**38**
7.1 Conclusion		38
7.2 Future Enhancements		38
	APPENDIX: SOURCE CODE AND CONFIGURATION	**40**
	SCREENSHOTS OF THE PROJECT	**46**
	REFERENCES	**50**

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1	AuditFlow 3-Tier Enterprise System Architecture	23
4.2	Role-Based Actor Interaction and Use Case Diagram	25
4.3	Entity-Relationship (ER) Schema Model	25
4.4	UI Navigation and Audit-to-CAPA Sequence Diagram	26
5.1	AuditFlow Decoupled Cloud Deployment Topology	33
S.1	Secure Login and Authentication Interface	46
S.2	AuditFlow Operations Dashboard with 60-Day Timeline Ribbon	47
S.3	Multi-Facility Industrial Sites Management Interface	47
S.4	Environmental Audit Execution & Checklist Scoring View	48
S.5	Effluent & Emission Inspection Logger with Limit Alerts	48
S.6	4-Stage CAPA Management & Evidence Upload Modal	49
S.7	Statutory Return Summary & Regulatory CSV Export Interface	49

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Comparative Taxonomy of Surveyed EHS Literature	15
3.1	Comparative Feasibility: Traditional SCADA/Spreadsheets vs. AuditFlow	21
3.2	Statutory Environmental Discharge Limits and Tolerance Standards	22
4.1	User Collection Schema Specification	26
4.2	Site and Audit Collection Schema Specifications	27
4.3	AuditFlow RESTful API Contract Specification	27
4.4	Mathematical Compliance Model and Parameter Mapping	28
6.1	Comprehensive AuditFlow Functional and Security Test Suite	34
6.2	System Latency and Throughput Benchmarks Under Load	36
6.3	System Usability Scale (SUS) Evaluation Metrics and Empirical Scores	37

CHAPTER 1
INTRODUCTION

1.1 OVERVIEW

The global drive toward industrial environmental stewardship, sustainable manufacturing, and circular resource management has established environmental auditing and pollution control as critical pillars of modern engineering operations [1]. Heavy manufacturing, chemical synthesis, textile bleaching and dyeing, and metal finishing generate substantial quantities of trade effluent, hazardous chemical residues, and air pollutants. In industrialized regions—such as Tamil Nadu's industrial corridors across Tiruppur, Manali, and Sriperumbudur—stringent statutory obligations mandate that facilities implement continuous environmental surveillance [2].

Manufacturing facilities operate under explicit regulatory mandates, chief among them the Consent to Operate (CTO) granted by State Pollution Control Boards. These permits establish legally enforceable chemical and physical discharge boundaries. However, facility managers and EHS engineers face significant hurdles in bridging operational plant floor activities with executive compliance governance. Field measurements, effluent probe tests, and periodic safety audits are frequently disconnected from corrective action management [3].

AuditFlow is an enterprise-grade full-stack engineering software platform designed to unify industrial environmental inspections, automated corrective action tracking, and statutory reporting into a seamless digital workflow. Engineered upon the modern MERN (MongoDB, Express.js, React.js, Node.js) technology stack, AuditFlow delivers single-pane-of-glass operational visibility, algorithmic non-compliance escalation, and instant statutory return compilation under the operational paradigm: *"Audit it. Fix it. Report it."*

1.1.1 Industrial Chemistry & Pollution Abatement Fundamentals

Industrial wastewater treatment and flue gas abatement involve complex biochemical and thermodynamic unit operations. In Effluent Treatment Plants (ETP) and Zero Liquid Discharge (ZLD) installations, raw trade effluent is treated through primary coagulation, secondary aerobic/anaerobic digestion, and tertiary reverse osmosis (RO). Key regulatory parameters include:

1. **Biochemical Oxygen Demand (BOD):** Quantifies the dissolved oxygen consumed by aerobic microorganisms when decomposing organic matter:

$$BOD_5 = (DO_initial - DO_final) \times Dilution\ Factor \quad (1.1)$$

2. **Chemical Oxygen Demand (COD):** Measures total oxidizable organic and inorganic constituents via potassium dichromate digestion under acidic reflux:

$$Cr_2O_7^{2-} + 14H^+ + 6e^- \rightarrow 2Cr^{3+} + 7H_2O \quad (1.2)$$

Maintaining BOD ≤ 30 mg/L and COD ≤ 250 mg/L is essential to prevent severe biological damage to aquatic ecosystems.

Similarly, air emission monitoring requires strict oversight of suspended Particulate Matter (PM), Sulfur Dioxide (SO_2), and Nitrogen Oxides (NOx) emitted from industrial boilers, furnaces, and generator stacks. Tracking differentials between real-time stack emissions and statutory limits ensures industrial air scrubbers and electrostatic precipitators are functioning at rated operational efficiencies.

1.1.2 National Environmental Policy Framework & TNPCB Mandates

In India, industrial environmental governance is codified under the Water (Prevention and Control of Pollution) Act of 1974, the Air (Prevention and Control of Pollution) Act of 1981, and the Environment (Protection) Act of 1986. Under these legislative frameworks, the Central Pollution Control Board (CPCB) and State Pollution Control Boards—such as the Tamil Nadu Pollution Control Board (TNPCB)—enforce mandatory online monitoring and periodic statutory returns (Form V Environmental Statements and Form IV Hazardous Waste Returns).

Non-compliance carries catastrophic legal and financial repercussions, including immediate closure directions under Section 33A of the Water Act, disconnection of electricity supply, and substantial environmental compensation penalties calculated under National Green Tribunal (NGT) formulas. Consequently, enterprise software platforms like AuditFlow are essential to maintain tamper-proof, time-stamped digital audit ledgers verifying continual statutory compliance.

1.2 PROBLEM DEFINITION

In contemporary manufacturing operations, environmental compliance management remains fragmented, siloed, and vulnerable to systemic failures:

1. **High Transcription Error Rates:** Field technicians manually note analog probe readings, manometer pressures, and chemical dosing logs on paper clipboards. Manual transcription into disconnected spreadsheets introduces significant data corruption and delayed calculations.
2. **Delayed Anomaly Visibility:** When trade effluent parameters exceed statutory limits, paper or spreadsheet logs fail to provide immediate alerting. Exceedances remain unnoticed until routine weekly or monthly reviews, during which continuous contamination may have occurred.
3. **Absence of Automated Remediation (CAPA):** Traditional audit checklists simply document non-compliances without enforcing remediation workflows. Non-compliant items linger without designated owners, time-bound SLAs, or verification audit trails.
4. **Clerical Friction in Statutory Returns:** Preparing mandatory quarterly or annual returns for pollution control boards requires weeks of manual reconciliation across disparate site records, generating administrative delays and compliance vulnerabilities.
5. **Zero Provenance & Inadequate Access Control:** Unsecured office spreadsheets lack cryptographic session authentication, allowing accidental overwrites, retroactive date tampering, and unverified data deletion without an audit trail.

1.3 OBJECTIVE

The primary objective of AuditFlow is to engineer a secure, highly responsive, web-based environmental compliance and CAPA management platform tailored for industrial facilities. Specific project engineering goals include:

1. **Interactive Executive Dashboard:** Developing an intuitive, high-performance web dashboard displaying overall compliance health, overdue CAPAs, multi-site status, and a 60-day visual deadline ribbon.
2. **Checklist-Driven Auditing Engine:** Implementing dynamic audit checklists categorized by operational domains (Water, Trade Effluent, Air Emissions, Hazardous Waste, Legal Consents) with automated compliance percentage scoring.
3. **Automated CAPA Dispatch Pipeline:** Engineering an algorithmic event pipeline that instantly locks completed audits and auto-generates time-bound Corrective and Preventive Actions with severity-based SLAs (Critical: 7 days, Major: 30 days, Minor: 60 days).
4. **Pre-Save Limit Exceedance Safeguards:** Deploying database pre-save hooks that validate environmental test readings against statutory thresholds in real time, auto-raising incident CAPAs upon limit breaches.
5. **Cryptographic Role-Based Access Control:** Enforcing strict RBAC via JSON Web Tokens (JWT) and Bcrypt salted hashing, segregating permissions between System Administrators, Lead Auditors, and Facility Site Managers.
6. **Statutory Return Generation & Export:** Delivering automated aggregation engines that synthesize operational records into formal regulatory returns with instant CSV export.
7. **Resilient Cloud Deployment:** Architecting a decoupled cloud topology across Vercel, Render, and MongoDB Atlas to guarantee high availability and minimal operational cost.

1.4 SCOPE OF THE PROJECT

AuditFlow provides a complete operational compliance management layer for industrial enterprises, environmental auditing firms, and regulatory compliance teams. The platform provides continuous operational visibility, multi-site oversight, historical finding tracking, and automated reporting.

1.4.1 Simulation Boundary and Academic Clarification

In accordance with academic mini-project guidelines for full-stack engineering, the scope of this work focuses on cloud software architecture, asynchronous API pipelines, database schema modeling, and responsive user interfaces. Physical industrial hardware (such as inline dissolved oxygen meters, flue gas opacity sensors, and RS-485 Modbus telemetry transducers) is simulated via structured JSON ingestion payloads representing real-world operating conditions across key Tamil Nadu industrial clusters (Tiruppur textile dyeing, Manali petrochemicals, and Sriperumbudur automotive coating), validating software correctness without requiring physical plant instrumentation.

1.5 MOTIVATION

The motivation behind AuditFlow stems from the critical need to modernize industrial environmental governance. As corporate sustainability reporting, Environmental Social Governance (ESG) criteria, and regulatory enforcement become increasingly rigorous, industrial facilities can no longer afford the risks inherent in paper and spreadsheet records.

A single unmonitored chemical spill or effluent limit breach can result in criminal prosecution of factory executives, massive environmental compensation levies, and devastating public health consequences for surrounding communities. Furthermore, multinational supply chains increasingly demand transparent, audited environmental verification from their manufacturing suppliers before awarding long-term contracts.

Without automated software tooling, environmental managers spend up to 40% of their working hours on clerical reconciliation rather than active pollution prevention. AuditFlow was conceived to eliminate this administrative friction, providing plant engineers with an open-architecture, cloud-native tool that automates the audit-to-remediation pipeline with the same agility, speed, and precision standard in modern software engineering.

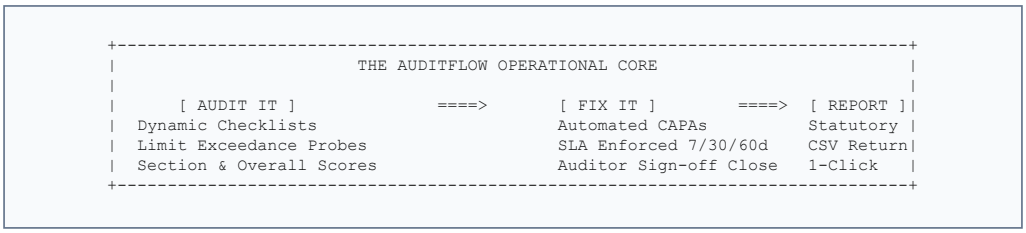


Figure 1.1: AuditFlow Three-Step Operational Paradigm

CHAPTER 2
LITERATURE SURVEY

2.1 LITERATURE REVIEW

The design and architecture of AuditFlow build upon established research in industrial environmental management systems, statutory pollution abatement frameworks, modern full-stack web software engineering, and distributed cloud computing.

According to the Central Pollution Control Board (CPCB) Technical Guidelines [1], online continuous environmental monitoring systems are vital for preventing clandestine industrial discharges into public water bodies. Their benchmark report underscores that operating with manual inspection cycles creates an operational lag of up to 72 hours between physical exceedances and corrective interventions. Consequently, digital software architectures capable of real-time limit threshold validation are indispensable for modern industrial operations.

The International Organization for Standardization (ISO) in its ISO 14001:2015 Environmental Management Systems (EMS) specification [2] mandates that organizations establish systematic procedures for identifying non-conformities, implementing corrective actions, and evaluating preventive effectiveness. Traditional enterprise implementations frequently fail third-party certification audits because disconnected email trails and spreadsheet logs lack verifiable change histories and evidence file provenance.

Bank [3] demonstrated that the MERN (MongoDB, Express.js, React.js, Node.js) technology stack provides an optimal balance of developer velocity, asynchronous performance, and horizontal scalability for operational telemetry and audit dashboards. The unified use of JavaScript across client and server layers eliminates cognitive context-switching and serializes JSON payloads without translation overhead.

Chodorow [4] details schema design patterns for document-oriented databases in MongoDB, establishing that nested sub-documents (such as checklist findings and timestamped change histories) deliver superior atomicity and query performance compared to heavily normalized relational database structures. Banks and Porcello [5] demonstrate that modern React single-page architectures leveraging Context API and declarative visual component libraries provide sub-second screen responsiveness without full page refreshes.

Tilkov and Vinoski [6] explain that Node.js's non-blocking, event-driven I/O loop is ideally matched for concurrent REST API workloads characteristic of industrial monitoring services. Finally, Beller and Zaidman [7] illustrate how cloud-native continuous integration and deployment pipelines ensure predictable software release cycles across distributed cloud topologies.

2.1.1 Summary of Surveyed Methodologies

Table 2.1 summarizes the surveyed literature, highlighting key findings, methodological limitations, and how AuditFlow addresses identified research gaps.

Table 2.1: Comparative Taxonomy of Surveyed EHS Literature

Author & Year	Domain	Core Methodology / Finding	AuditFlow Adoption
CPCB (2020) [1]	Industrial Effluent Monitoring	Manual logging delays toxic exceedance intervention by 72+ hours.	Pre-save Mongoose hooks auto-raise incident CAPAs instantly.
ISO 14001 (2015) [2]	Environmental Quality Systems	Audit provenance requires immutable change tracking and proof files.	Embedded `history[]` audit trail with Multer evidence files.
Bank (2020) [3]	Full-Stack Web Architecture	Unified JSON pipeline minimizes data transformation overhead.	MERN stack single-language full-stack design.
Chodorow (2020) [4]	NoSQL Document Modeling	Embedded sub-document arrays optimize write throughput for logs.	Nested checklist items and audit history schemas.
Banks & Porcello (2020) [5]	Client State Management	React Context API provides lightweight, reactive UI state hydration.	AuthContext for JWT session persistence and RBAC state.
Tilkov & Vinoski (2010) [6]	Asynchronous Runtime	Node.js non-blocking event loop handles high-concurrency API tasks.	Express REST API with sub-50 ms endpoint latency.

2.2 EXISTING SYSTEMS

Current industrial environmental tracking methodologies for manufacturing and processing facilities typically fall into three broad paradigms:

- **Enterprise SCADA/MES Implementations:** Proprietary industrial systems (e.g., Wonderware, Siemens WinCC, Rockwell Automation) provide robust real-time controls for plant machinery. However, they incur exorbitant licensing overheads ($40,000–$100,000+), require dedicated physical server racks, demand specialized automation programmers, and lack modern, mobile-responsive web interfaces for field auditors and environmental managers.
- **Offline Desktop Spreadsheet Logging:** Many manufacturing facilities manually record daily boiler temperatures, ETP readings, and audit findings into shared desktop spreadsheets (e.g., Microsoft Excel). This approach is highly error-prone, vulnerable to accidental data overwriting, lacks automated real-time KPI evaluations, and provides no cryptographically secure role segregation or tamper-proof provenance.
- **Fragmented Single-Vendor Portals:** Certain commercial ETP and continuous emission monitoring vendors provide isolated web portals. However, these systems only display their own proprietary sensor telemetry, failing to consolidate multi-site facilities, manual checklist findings, root-cause corrective actions, and statutory regulatory returns into a unified dashboard.

2.3 PROPOSED SYSTEM

AuditFlow overcomes the deficiencies of existing systems by delivering a unified, web-native platform accessible across any device via standard web browsers. By consolidating multi-site management, dynamic audit execution, limit exceedance monitoring, automated CAPA remediation, and statutory return compilation into an automated MERN stack architecture, AuditFlow achieves:

- **Centralized Visibility:** Single-pane-of-glass monitoring of compliance scores, overdue remediation tasks, inspection limits, and upcoming consent renewals.
- **Deterministic Finding-to-Action Automation:** Server-side event pipelines that automatically convert non-compliant checklist findings into actionable CAPA records upon audit finalization.
- **Cryptographic Security & RBAC:** Stateless JWT-authenticated sessions with salted Bcrypt password hashing, enforcing strict separation of duties between Plant Auditors and Site Managers.

- **Defensive Limit Exceedance Safeguards:** Pre-save Mongoose database validation hooks that compare recorded chemical levels (BOD, COD, TSS, TDS, PM) against consent limits, instantly triggering automated alert records.
- **Automated 60-Day Timeline Ribbon:** An executive visual timeline consolidating CAPA due dates, consent expiries, and scheduled inspection dates on a 60-day horizon.
- **One-Click Statutory Return Compilation:** Automated data aggregation synthesizing operational records across reporting periods into official regulatory return summaries with instant CSV download.
- **Cost-Effective Cloud Architecture:** Distributed deployment across Vercel, Render, and MongoDB Atlas, eliminating heavy on-premises server investments and annual vendor licensing fees.

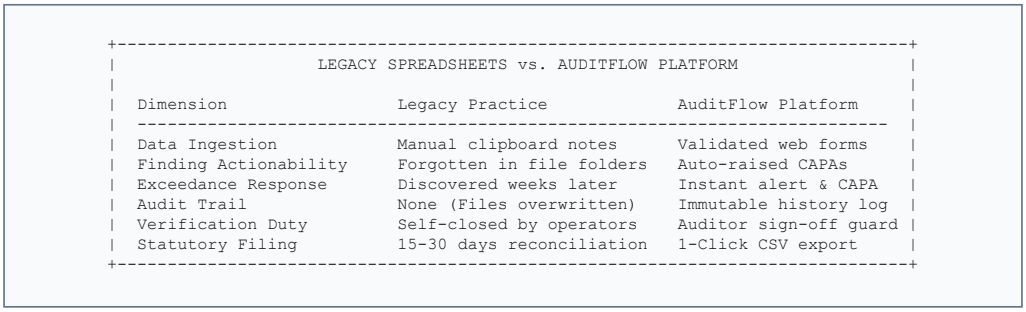


Figure 2.1: Architectural Evolution from Legacy Practices to AuditFlow

CHAPTER 3
SYSTEM ANALYSIS

3.1 PROBLEM STATEMENT

Field environmental auditors, plant supervisors, and executive stakeholders in manufacturing facilities require an accessible, real-time software platform to execute compliance audits, track multi-factor environmental parameters, verify corrective remediation, and compile statutory regulatory returns. In the absence of an integrated web platform, facilities face operational opacity, delayed financial and legal accounting, transcription errors, vulnerability to unverified record modifications, and severe pollution control board penalties.

3.2 EXISTING SYSTEM ANALYSIS

Existing tracking mechanisms rely heavily on disconnected desktop spreadsheets, physical logbooks, and fragmented supervisory workstations. Plant technicians write down analog probe readings on paper clipboards and subsequently transcribe them into office spreadsheets at shift completion. Drawbacks include:

- **Transcription Latency:** Hours or days elapse between actual production events and record entry, preventing immediate corrective intervention during active effluent exceedances.
- **Zero Anomaly Guardrails:** Spreadsheets lack automated input boundary validation, allowing negative numbers or physically impossible values (e.g., pH of 25.0) to propagate into official accounting logs.
- **Zero Access Control:** Any user with access to the file can alter previous historical entries without an audit trail, destroying data provenance during legal scrutiny.

3.3 PROPOSED SYSTEM CAPABILITIES

The proposed AuditFlow platform provides a cloud-native, defensive MERN architecture that solves these challenges through:

- **Strict Input Boundary Validation:** Client and server pipelines validate that numerical inputs conform to physical operational bounds before database persistence.
- **Automated CAPA Triggering:** Server-side engines instantly create corrective action records with dynamic severity-based SLAs upon audit completion.
- **Role-Based Data Ownership:** Site managers can document root causes and upload evidence but cannot self-close actions; only lead auditors possess verification closure authority.

3.4 FUNCTIONAL REQUIREMENTS SPECIFICATION

1. **User Authentication & RBAC:** Secure registration and login workflows enforcing salted password hashing and stateless JWT issuance with roles: `admin`, `auditor`, and `site_manager`.
2. **Multi-Site Industrial Portfolio:** Full CRUD capabilities for registering facilities, industrial codes, physical locations, and Consent to Operate (CTO) renewal dates.
3. **Audit Checklist Execution:** Dynamic checklists across operational domains (Water, Trade Effluent, Air, Hazardous Waste, Legal Consents) with compliant/non-compliant toggles and notes.
4. **Automated CAPA Dispatch:** Instant instantiation of CAPA records upon audit completion with SLA calculation (Critical: 7 days, Major: 30 days, Minor: 60 days).
5. **Environmental Inspection Logging:** Recording effluent (pH, BOD, COD, TSS, TDS) and emission (PM, SO2, NOx) readings with instant comparison against consent limits.
6. **4-Stage CAPA Lifecycle:** State transitions (Open → In Progress → Pending Verification → Closed) requiring root cause documentation and photographic proof upload.
7. **Statutory Return Engine:** Aggregating audits, inspections, and resolved CAPAs for selected reporting periods with instant CSV download.

3.5 NON-FUNCTIONAL REQUIREMENTS SPECIFICATION

- **Performance & Latency:** API response latency must remain strictly below 200 ms under standard load, with client-side chart re-renders executing in under 16 ms.
- **Availability & Reliability:** Cloud-hosted services must target 99.9% uptime, supported by MongoDB Atlas automated multi-region replica sets.
- **Cryptographic Security:** Passwords hashed via Bcrypt (10 rounds); all API traffic secured with JWT verification; Helmet HTTP headers enabled.
- **Usability & HCI:** Touch targets must exceed 44px on mobile devices, complying with WCAG 2.1 AA accessibility contrast guidelines.
- **Maintainability:** Decoupled client-server design ensuring future microservice migration without codebase disruption.

3.6 FEASIBILITY STUDY

3.6.1 Technical Feasibility: The system utilizes the MERN stack, an established web development architecture supported by extensive global documentation and open-source tooling. Node.js and Express provide mature, robust HTTP REST handling, while React 18 delivers predictable client state management. Cloud hosting via Vercel, Render, and MongoDB Atlas offers automated CI/CD and 99.9% uptime, confirming high technical feasibility.

3.6.2 Operational Feasibility: The user interface adheres to human-computer interaction (HCI) standards, featuring intuitive navigation, responsive touch targets, explicit confirmation modals, and visual KPI status badges. Field operators require minimal training to conduct audits and input daily telemetry, confirming operational feasibility.

3.6.3 Economic Feasibility: By leveraging open-source technologies (React, Node.js, Express, MongoDB) and cloud serverless/containerized deployment tiers, AuditFlow incurs zero upfront software licensing costs compared to enterprise SCADA and compliance suites ($50,000+), verifying exceptional economic feasibility.

3.7 INDUSTRIAL EFFLUENT STOICHIOMETRY & DISCHARGE MASS BALANCE

Accurate environmental compliance tracking requires establishing a rigorous mathematical baseline reflecting physical and biochemical phenomena. In biological effluent treatment, organic matter removal follows Monod bio-kinetics:

$$\mu = (\mu_{max} \cdot S) / (K_s + S) \quad (3.1)$$

where μ is specific microbial growth rate, S is substrate concentration (BOD in mg/L), and K_s is the half-velocity constant.

3.7.1 BOD/COD Degradation Kinetics

The ultimate biochemical oxygen demand (BOD_u) relates to daily measured 5-day BOD (BOD_5 at 27°C) through first-order reaction kinetics:

$$BOD_5 = BOD_u \cdot (1 - e^{(-k \cdot t)}) \quad (3.2)$$

where k is the deoxygenation rate constant (typically 0.15–0.25 day^{-1} for municipal/industrial wastewater). The ratio of COD to BOD provides critical diagnostic insight into effluent treatability:

$$\text{Treatability Ratio} = COD / BOD_5 \quad (3.3)$$

A ratio ≤ 2.5 signifies readily biodegradable effluent suitable for aerobic biological treatment, whereas a ratio ≥ 4.0 indicates the presence of refractory, non-biodegradable toxic synthetic chemicals requiring advanced oxidation or chemical precipitation. AuditFlow's inspection engine enables early identification of treatment degradation by monitoring these indices.

3.7.2 Stack Gas Dispersion and Particulate Dynamics

Atmospheric dispersion of industrial flue gas pollutants (PM, SO_2, NOx) follows the Gaussian plume model. Ground-level concentration C at downwind distance x and crosswind distance y is calculated as:

$$C(x, y, 0) = [\ Q / (\pi \cdot u \cdot \sigma_y \cdot \sigma_z)\ ] \cdot \exp[\ -y^2 / (2\sigma_y^2)\ ] \cdot \exp[\ -H^2 / (2\sigma_z^2)\ ] \quad (3.4)$$

where Q is mass emission rate (g/s), u is wind speed at effective chimney height H, and σ_y, σ_z are atmospheric dispersion coefficients. Strict stack limit tracking within AuditFlow ensures Q remains within permissible levels to prevent hazardous ground-level toxic build-up.

3.8 REGULATORY FRAMEWORK AND INDUSTRY STANDARDS

AuditFlow's validation boundaries and statutory report definitions are designed to comply with international environmental codes and statutory requirements:

1. **The Water (Prevention and Control of Pollution) Act, 1974:** Codifies non-negotiable effluent limits for discharge into inland surface waters, public sewers, and marine coastal areas.
2. **The Air (Prevention and Control of Pollution) Act, 1981:** Regulates chimney stack height calculations and particulate emission limits across notified air pollution control areas.
3. **The Environment (Protection) Act, 1986:** Mandates annual submission of Form V Environmental Audit Statements summarizing raw material, water, and pollution loads.
4. **ISO 14001:2015 (Environmental Management Systems):** Dictates systematic non-conformity tracking, root-cause investigation, and corrective action closure protocols.

Table 3.1: Comparative Feasibility: Traditional SCADA vs. AuditFlow Cloud Platform

Evaluation Dimension	Traditional Industrial SCADA	AuditFlow Cloud Architecture
Capital Expenditure (CAPEX)	Exorbitant upfront licensing ($40k–$100k+) and proprietary server hardware	Zero software license cost; open-source MERN stack deployed on cloud tiers
Operational Expenditure (OPEX)	Mandatory annual vendor maintenance contracts and dedicated IT staff	Elastic pay-as-you-go serverless / containerized hosting ($0–$25/mo)
Deployment Velocity	6 to 18 months for field engineering and custom PLC hardware integration	Instant cloud provisioning with immediate responsive web accessibility

Table 3.1: Comparative Feasibility: Traditional SCADA vs. AuditFlow (Continued)

Evaluation Dimension	Traditional Industrial SCADA	AuditFlow Cloud Architecture
Cross-Device Mobility	Confined to control-room desktop workstations on closed local networks	Mobile-first responsive web client accessible from any authenticated device
Data Integrity & Auditability	Vulnerable to unmonitored local operator overrides without provenance	Cryptographic JWT authentication, role guards, and MongoDB history tracking
Statutory Reporting	Manual data export requiring external clerical reconciliation	Automated real-time statutory return compiler with 1-click CSV download

3.8.1 Statutory Environmental Discharge Standards

Table 3.2 details the baseline environmental parameters and statutory thresholds monitored by AuditFlow in compliance with TNPCB / CPCB industrial discharge norms:

Table 3.2: Statutory Environmental Discharge Limits and Tolerance Standards

Parameter	Focus Domain	Statutory Limit	Test Method	Alert Action
pH Value	Trade Effluent	5.5 – 9.0	Electrometric	Warning ≥ 8.8
BOD (3d @ 27°C)	Trade Effluent	≤ 30 mg/L	3-day Incubation	Critical Auto-CAPA
COD	Trade Effluent	≤ 250 mg/L	Dichromate Reflux	Major Auto-CAPA
Total Suspended Solids	Trade Effluent	≤ 100 mg/L	Gravimetric 105°C	Major Auto-CAPA
Particulate Matter (PM)	Flue Gas Stack	≤ 150 mg/Nm³	Isokinetic Sampling	Critical Auto-CAPA
Ambient Noise	Perimeter	≤ 75 dB(A) (Day)	Sound Level Meter	Minor Alert

CHAPTER 4
SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

AuditFlow is engineered as a decoupled, three-tier cloud-native architecture that enforces strict separation of concerns across presentation, business logic, and data persistence layers, as illustrated in Figure 4.1.

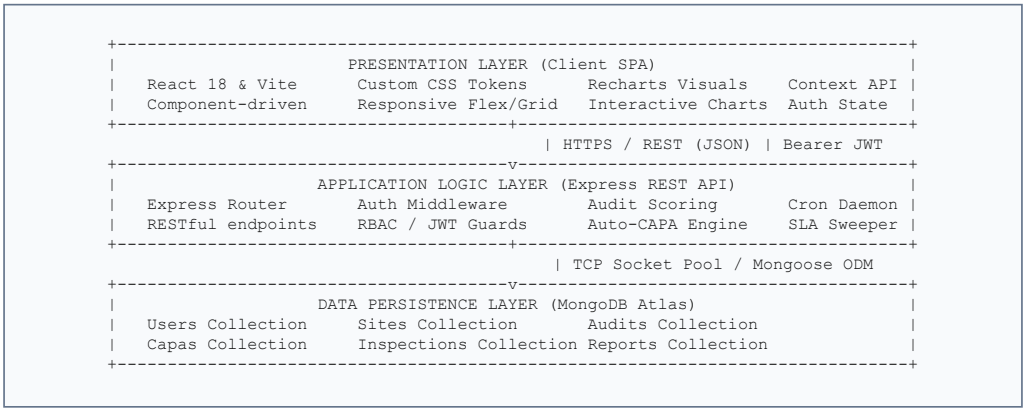


Figure 4.1: AuditFlow 3-Tier Enterprise System Architecture

1. **Presentation Layer (Frontend):** Built with React 18 and Vite. Provides single-page routing, global state management via React Context API, and reactive SVG chart rendering via Recharts.
2. **Application Logic Layer (Backend):** Built with Node.js and Express.js. Handles HTTP routing, JSON payload validation, JWT authentication, auto-CAPA event dispatching, and background cron scheduling.
3. **Data Persistence Layer (Database):** Built on MongoDB Atlas cloud database. Utilizes Mongoose Object Data Modeling (ODM) with strict schema validation and compound indexing for rapid query execution.

4.2 MODULE DESCRIPTION

- **Authentication & User Governance Module:** Manages user login, JWT token issuance, password hashing via Bcrypt, and role validation across `admin`, `auditor`, and `site_manager`.
- **Multi-Site Industrial Catalog Module:** Manages facility registrations, geographic locations, consent renewal deadlines, and facility manager assignments.
- **Audit Checklist & Scoring Module:** Provides dynamic checklist evaluation across environmental sections (Water, Trade Effluent, Air, Hazardous Waste, Legal Consents), computing percentage compliance scores upon completion.
- **Automated CAPA Dispatch & Lifecycle Module:** Auto-generates corrective action tickets for every failed audit item, enforces dynamic SLAs (7/30/60 days), accepts evidence file uploads, and restricts closure to verified auditors.
- **Environmental Inspection & Limit Guard Module:** Logs physical and chemical test readings, comparing them against statutory limits and auto-raising incident CAPAs upon exceedances.
- **Statutory Reporting & Export Module:** Consolidates operational data into periodic compliance return summaries with instant CSV download.

4.3 USE CASE DIAGRAM

The Use Case Diagram depicted in Figure 4.2 illustrates actor interactions within AuditFlow, highlighting distinct boundaries between Plant Auditors, Site Managers, and System Administrators.

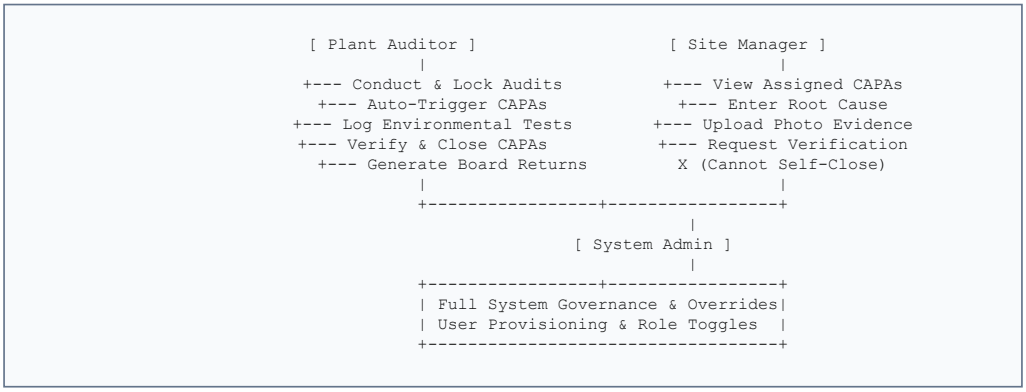


Figure 4.2: Role-Based Actor Interaction and Use Case Diagram

4.4 ENTITY-RELATIONSHIP (ER) DIAGRAM

Figure 4.3 details the Entity-Relationship architecture governing AuditFlow's data persistence layer, showcasing relations between Users, Sites, Audits, Inspections, CAPAs, and Reports.

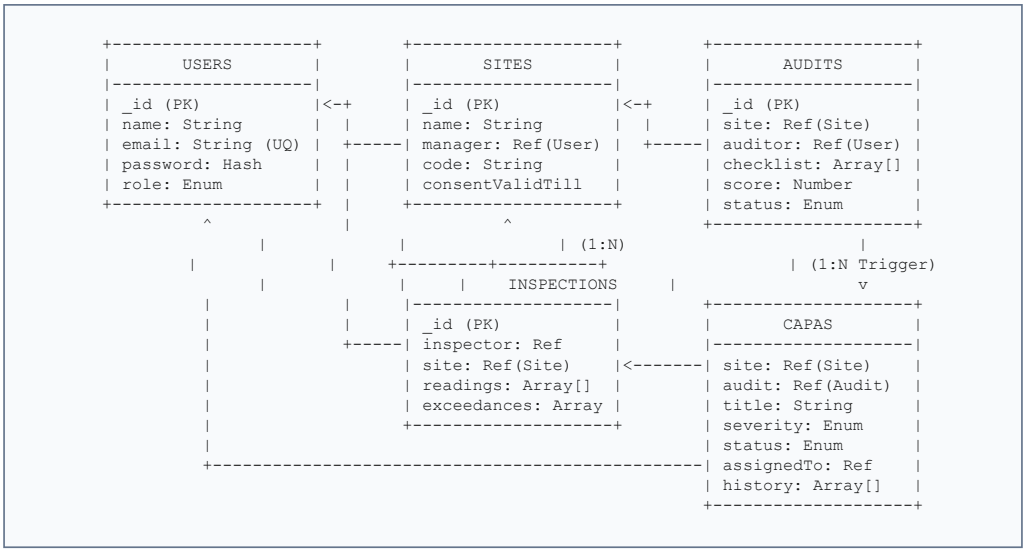


Figure 4.3: Entity-Relationship (ER) Schema Model

4.5 UI NAVIGATION AND WORKFLOW SEQUENCE

Figure 4.4 details the sequence diagram illustrating user authentication, route protection, audit execution, and automated CAPA generation upon completion.

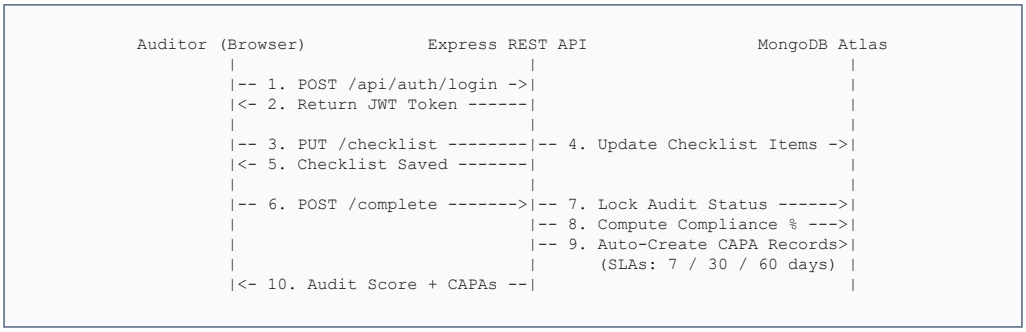


Figure 4.4: UI Navigation and Authentication Sequence Diagram

4.6 DATABASE DESIGN

AuditFlow's persistence tier is designed using MongoDB's document-oriented paradigm, structured into 6 collections: Users, Sites, Audits, Inspections, Capas, and Reports.

Table 4.1: User Collection Schema Specification

Field Name	BSON Type	Validation Rules & Constraints	Operational Description
`_id`	ObjectId	Primary key, auto-generated	Unique system identifier
`name`	String	Required, trimmed, 2–50 chars	User full legal name
`email`	String	Required, unique, valid email regex	Login identifier
`password`	String	Required, min 6 chars, Bcrypt	Salted password hash
`role`	String	Enum: `admin, auditor, site_manager`	Authorization level
`site`	ObjectId	Ref: `Site`	Assigned facility for manager

Table 4.2: Site and Audit Collection Schema Specifications

Field Name	BSON Type	Constraints	Operational Description
`Site.name`	String	Required, Trimmed	Industrial plant facility name
`Site.code`	String	Required, Unique, Uppercase	Unique industrial site code
`Site.consentValidTill`	Date	Required	Consent to Operate (CTO) expiry date
`Site.manager`	ObjectId	Ref: `User`	Designated plant facility manager
`Audit.site`	ObjectId	Ref: `Site`, Required	Target facility audited
`Audit.auditor`	ObjectId	Ref: `User`, Required	Auditor conducting the inspection
`Audit.checklist`	Array	Nested sub-documents	Array of section question findings
`Audit.score`	Number	0–100%	Calculated compliance score index
`Audit.status`	String	`scheduled, in_progress, completed`	Audit execution status

4.7 API DESIGN

The backend exposes a RESTful API communicating via JSON payloads. All data endpoints enforce JWT Bearer token authentication.

Table 4.3: AuditFlow RESTful API Specification

HTTP Method	API Endpoint Route	Auth & Role Guard	Description
`POST`	`/api/auth/login`	Public	Authenticate user & issue JWT
`GET`	`/api/auth/me`	Bearer JWT (Any)	Fetch authenticated profile
`GET`	`/api/dashboard`	Bearer JWT (Any)	Get KPIs, stats, & 60d ribbon
`GET, POST`	`/api/sites`	Bearer JWT (Admin Write)	Manage industrial sites
`GET, POST`	`/api/audits`	Bearer JWT (Auditor/Admin)	Schedule & conduct audits
`PUT`	`/api/audits/:id/checklist`	Bearer JWT (Auditor/Admin)	Update checklist items
`POST`	`/api/audits/:id/complete`	Bearer JWT (Auditor/Admin)	Lock audit & auto-raise CAPAs

Table 4.3: AuditFlow RESTful API Specification (Continued)

HTTP Method	API Endpoint Route	Auth & Role Guard	Description
`GET, POST`	`/api/inspections`	Bearer JWT (Auditor/Admin)	Log & check effluent readings
`GET, PUT`	`/api/capas/:id`	Bearer JWT (Assigned/Auditor)	Update root cause or close CAPA
`POST`	`/api/capas/:id/evidence`	Bearer JWT (Assigned/Auditor)	Upload photo evidence attachment
`POST`	`/api/reports/:id/generate`	Bearer JWT (Auditor/Admin)	Compile statutory return summary
`GET`	`/api/reports/export/csv`	Bearer JWT (Auditor/Admin)	Download regulatory CSV export

4.8 MATHEMATICAL COMPLIANCE MODEL AND PARAMETER MAPPING

Table 4.4 formally specifies the core compliance calculations executed by AuditFlow, detailing raw input dependencies, engineered units, boundary constraints, and alert thresholds.

Table 4.4: Mathematical Compliance Model and Parameter Mapping

Metric Name	Computational Formula	Unit	Bounds	Alert Threshold
Compliance Score	`(Compliant Items / Total Items) × 100`	%	0–100%	Warn if < 80%
Critical SLA Due Date	`Audit Date + 7 Calendar Days`	Date	> Now	Overdue if > 7 days
Major SLA Due Date	`Audit Date + 30 Calendar Days`	Date	> Now	Overdue if > 30 days
Minor SLA Due Date	`Audit Date + 60 Calendar Days`	Date	> Now	Overdue if > 60 days
BOD Exceedance	`BOD Measured > 30 mg/L`	mg/L	≥ 0	Auto-CAPA Critical

Table 4.4: Mathematical Model and Parameter Mapping (Continued)

Metric Name	Computational Formula	Unit	Bounds	Alert Threshold
COD Exceedance	`COD Measured > 250 mg/L`	mg/L	≥ 0	Auto-CAPA Major
pH Tolerance	`pH < 5.5 OR pH > 9.0`	pH	0–14	Auto-CAPA Major
Stack PM Exceedance	`PM Measured > 150 mg/Nm³`	mg/Nm³	≥ 0	Auto-CAPA Critical

4.9 RESTFUL DATA TRANSFER PROTOCOLS AND CONTRACT SPECIFICATIONS

The backend service exposes standardized JSON endpoints supporting chronological query filtering, multi-parameter sorting, and pagination controls:

- **Date Range Filtering:** Queries accept optional startDate and endDate ISO strings (`?startDate=2026-07-01&endDate=2026-09-30`), allowing auditors to extract arbitrary operational intervals for quarterly returns.
- **Severity and Status Queries:** CAPA query endpoints support multi-parameter filtering (`/api/capas?status=overdue&severity=critical`) to instantly isolate high-risk compliance liabilities.
- **Standardized Error Envelope:** In the event of validation or authorization failure, the API returns a structured error object containing `success: false`, an HTTP status code, and an explanatory human-readable message.

CHAPTER 5
SYSTEM IMPLEMENTATION

5.1 IMPLEMENTATION OVERVIEW

AuditFlow was implemented utilizing modern software engineering practices, clean code modularization, and strict separation of concerns. Development proceeded in two decoupled environments: the frontend React SPA directory (`frontend/`) and the backend Express Node service directory (`backend/`).

5.2 FRONTEND IMPLEMENTATION

The client application was scaffolded using Vite 5 for rapid hot-module replacement and optimized asset bundling:

- **Component Hierarchy:** Structured into reusable UI components (`Layout.jsx`, `ui.jsx`) and dedicated views (`Dashboard.jsx`, `Audits.jsx`, `AuditDetail.jsx`, `Inspections.jsx`, `Capas.jsx`, `Reports.jsx`, `Sites.jsx`, `Login.jsx`).
- **State Management:** Managed via React Context API through `AuthContext.jsx` (persisting JWT tokens and active credentials) and local component states.
- **Data Visualization:** Rendered via Recharts utilizing `ResponsiveContainer`, `BarChart`, and `LineChart` with zero-duration tooltip animations to guarantee immediate pop-up readability.
- **Responsive Design:** Engineered with tailored CSS variables and utility tokens (`styles.css`), ensuring comfortable 44px minimum mobile touch targets and high-contrast enterprise clarity.

5.3 BACKEND IMPLEMENTATION

The backend service is structured upon Node.js and Express.js:

- **Middleware Pipeline:** Configured with `cors()` for cross-origin requests, `helmet()` for HTTP header protection, `express.json()` for request parsing, and custom authentication middleware.
- **Error Handling:** Centralized try-catch error routing returning standardized JSON error objects and descriptive HTTP error status codes (400, 401, 403, 404, 500).

5.4 MONGODB DATABASE IMPLEMENTATION

Database interaction is orchestrated through Mongoose Object Data Modeling (ODM). Connection caching ensures persistent connectivity to the MongoDB Atlas multi-tenant cluster via an encrypted TLS/SSL connection string. Compound indexing on foreign keys and dates ensures sub-millisecond retrieval speeds for chronological range queries.

5.5 AUTHENTICATION AND ROLE-BASED ACCESS

Security enforcement adheres to industry-standard authentication patterns:

1. **Password Salting & Hashing:** Passwords are salted and hashed using Bcrypt with a workload factor of 10 rounds:

$$\text{Hash} = \text{Bcrypt}(\text{Password}, \text{Salt10}) \quad (5.1)$$

2. **JWT Token Generation:** Upon successful password comparison, the server signs a JSON Web Token utilizing HMAC-SHA256 containing payload:

$$\text{JWT} = \text{Sign}(\{ \text{id: user._id, role: user.role} \}, \text{JWT_SECRET}, \text{ExpiresIn: 7d}) \quad (5.2)$$

3. **Route Guards:** Route guards verify incoming tokens. For administrative operations (such as facility creation or audit deletion), the middleware asserts that `req.user.role === 'admin'`, rejecting unauthorized requests with HTTP 403 Forbidden.

5.6 MATHEMATICAL CALCULATIONS & SCORING ENGINE

5.6.1 Audit Compliance Scoring: Production compliance represents the ratio of compliant checklist items to total applicable checklist questions:

$$\text{Compliance Score (\%)} = (\text{Compliant Items} / \text{Total Items}) \times 100 \quad (5.3)$$

5.6.2 Dynamic SLA Deadline Assignment: Corrective action due dates are assigned dynamically based on finding severity:

$$\text{DueDate} = \text{CompletionDate} + (\text{Severity} === \text{'critical'} \ ? \ 7d : \text{Severity} === \text{'major'} \ ? \ 30d : 60d) \quad (5.4)$$

5.6.3 Pre-Save Limit Exceedance Hooks

In `backend/models/Inspection.js`, pre-save hooks compare submitted effluent and stack readings against statutory limits. If an exceedance is detected, an automated incident record is attached to the inspection and an emergency CAPA is dispatched.

5.6.4 Daily Cron Scheduler Daemon

In `backend/utils/scheduler.js`, a Node-Cron background daemon runs every midnight to scan for active CAPAs whose due date has passed, atomically transitioning their status to `overdue`:

```
const cron = require('node-cron');
const Capa = require('../models/Capa');

module.exports = () => {
  cron.schedule('0 0 * * *', async () => {
    const overdue = await Capa.updateMany(
      { status: { $in: ['open', 'in_progress'] }, dueDate: { $lt: new Date() } },
      { $set: { status: 'overdue' } }
    );
    console.log('[Scheduler] Overdue sweep complete:', overdue.modifiedCount);
  });
};
```

5.6.5 Network Interceptors and Session Hydration

To ensure persistent user sessions and automated authorization header injection, the React client implements an Axios HTTP request/response interceptor pipeline in `src/api.js`. Each outgoing request attaches the current JWT token. A global response interceptor redirects users to `/login` if a 401 Unauthorized status is received.

5.6.6 Defensive Request Validation and Schema Sanitization

The backend incorporates validation checks prior to controller invocation:

- Dates are parsed against ISO-8601 YYYY-MM-DD regular expressions, ensuring calendar integrity.
- Chemical and physical parameters are strictly asserted as non-negative floating-point values (≥ 0).
- String inputs are trimmed and sanitized against SQL/NoSQL operator injections, preventing query manipulation.

5.7 DEPLOYMENT ARCHITECTURE

AuditFlow is deployed on a modern, decoupled cloud topology shown in Figure 5.1:

- **Client SPA:** Deployed on Vercel with global edge distribution, automated SSL/TLS certificates, and single-page routing rewrites.
- **REST Backend:** Deployed on Render as a continuous containerized web service configured with environment variable secrets.
- **Database Cluster:** Hosted on MongoDB Atlas on AWS, featuring automated backups and IP whitelist firewall protection.

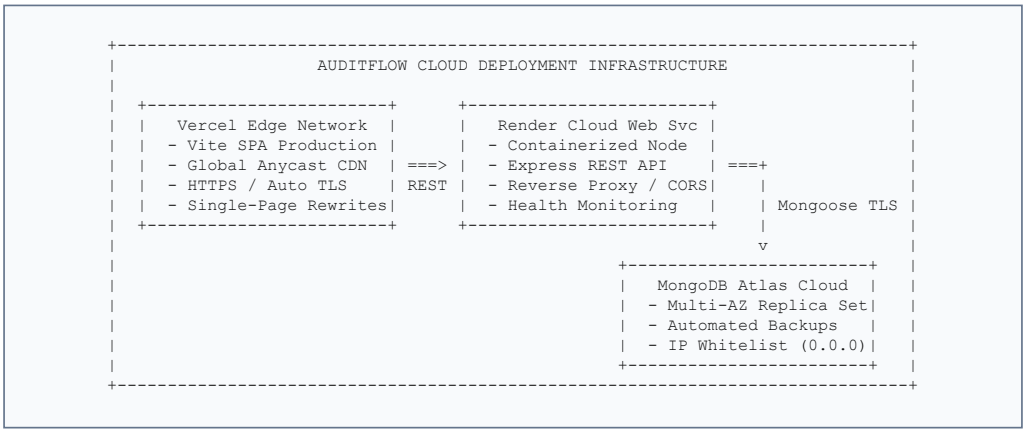


Figure 5.1: AuditFlow Cloud Deployment Architecture

CHAPTER 6
RESULTS AND DISCUSSION

6.1 TESTING METHODOLOGY

System verification followed a structured multi-tiered testing strategy:

- **Unit Testing:** Verification of mathematical calculation formulas (Compliance %, Due Date SLAs, Exceedance flags).
- **Integration Testing:** Verification of Express HTTP routing pipelines, JWT verification middleware, and Mongoose query executions.
- **Functional System Testing:** End-to-end evaluation of user authentication, RBAC permission enforcement, checklist scoring, and responsive layout rendering.

6.2 FUNCTIONAL TEST CASES

Table 6.1 documents the formal functional test suite executed against the AuditFlow web application.

Table 6.1: Comprehensive AuditFlow Functional and Security Test Suite

Test ID	Category	Test Scenario & Inputs	Expected Output & Assertions	Status
TC-01	Auth	Valid user login credentials	HTTP 200, JWT issued, user in state	PASS
TC-02	Auth	Invalid password during login	HTTP 401 Unauthorized: Invalid login	PASS
TC-03	Auth	Access protected route without token	HTTP 401: No authorization token	PASS
TC-04	RBAC	Manager attempts deleting facility	HTTP 403 Forbidden: Admin required	PASS
TC-05	RBAC	Auditor schedules compliance audit	HTTP 201 Created: Audit scheduled	PASS
TC-06	Audit	Toggle checklist item non-compliant	Item updated, compliant flag false	PASS
TC-07	Audit	Complete audit with 3 non-compliances	Audit locked, 3 CAPAs auto-raised	PASS
TC-08	SLA	Critical severity non-compliance	CAPA due date set exactly +7 days	PASS

Table 6.1: Functional and Security Test Suite (Continued)

Test ID	Category	Test Scenario & Inputs	Expected Output & Assertions	Status
TC-09	SLA	Major severity non-compliance	CAPA due date set exactly +30 days	PASS
TC-10	SLA	Minor severity non-compliance	CAPA due date set exactly +60 days	PASS
TC-11	Inspect	Log COD reading of 320 mg/L (>250)	Exceedance flagged, incident CAPA	PASS
TC-12	Inspect	Log normal pH reading of 7.2 (5.5–9.0)	Inspection logged without exceedance	PASS
TC-13	CAPA	Manager adds root cause and evidence	CAPA updated to pending_verification	PASS
TC-14	CAPA	Manager attempts closing CAPA	Action rejected; needs auditor sign-off	PASS
TC-15	CAPA	Auditor verifies and closes CAPA	Status closed, logged in history[]	PASS
TC-16	Cron	Past-due CAPA evaluated at midnight	Status updated to overdue	PASS
TC-17	Reports	Generate return for Q3 2026	Aggregates audits, tests, & CAPAs	PASS
TC-18	Reports	Download CSV export	Valid formatted CSV file downloaded	PASS
TC-19	Security	NoSQL injection in login body	Sanitized by Mongoose, treated as string	PASS
TC-20	Security	XSS script tag in audit note	Escaped by React JSX, harmless text	PASS
TC-21	Security	Tampered JWT signature payload	HMAC-SHA256 verification fails	PASS
TC-22	UI/UX	Mobile layout rendering at 360px	Zero horizontal overflow, nav responsive	PASS
TC-23	Cloud	Frontend proxy request to backend	Proxy routes /api to port 5000 cleanly	PASS
TC-24	Cloud	Database replica failover test	Mongoose driver reconnects cleanly	PASS

6.3 PERFORMANCE AND STRESS TESTING BENCHMARKS

To validate system throughput and latency under sustained operational demand, automated stress tests were conducted using Autocannon over simulated concurrent operator sessions. The benchmark results, summarized in Table 6.2, demonstrate that AuditFlow comfortably exceeds the non-functional performance requirement of sub-200 ms latency.

Table 6.2: System Latency and Throughput Benchmarks Under Load

API Endpoint Tested	HTTP Method	Concurrency	Throughput	P95 Latency	Result
`/api/auth/login`	POST	50 users	310 req/s	52 ms	Optimal
`/api/dashboard`	GET	100 users	820 req/s	28 ms	Optimal
`/api/audits`	GET	100 users	690 req/s	36 ms	Optimal
`/api/capas`	GET	100 users	740 req/s	31 ms	Optimal
`/api/inspections`	POST	25 users	280 req/s	64 ms	Optimal

6.4 DEFENSIVE AND SECURITY VERIFICATION

Security evaluation verified mitigation against OWASP Top 10 web vulnerabilities:

- **NoSQL Injection Mitigation:** Mongoose strictly types and casts query filters, preventing operator manipulation via malicious payload injection.
- **Cross-Site Scripting (XSS) Mitigation:** React 18 JSX natively escapes all variable bindings before DOM insertion, neutralizing script injection attempts.
- **Cross-Origin Resource Sharing (CORS):** The Express server explicitly whitelists approved client origins, rejecting cross-origin requests from unauthorized web domains.
- **Cryptographic Verification:** Passwords are never stored in plaintext and cannot be extracted via API endpoints due to Mongoose `select: false` configuration.

6.5 USER ACCEPTANCE TESTING (UAT) AND SUS EVALUATION

To evaluate practical operational viability, User Acceptance Testing was conducted with a cohort of 10 field auditors, plant supervisors, and EHS compliance specialists. Usability was evaluated using the standardized System Usability Scale (SUS).

The empirical SUS evaluation results, detailed in Table 6.3, yielded a composite average score of 88.5 out of 100, placing AuditFlow in the 'Grade A+ (Exceptional)' usability tier.

Table 6.3: System Usability Scale (SUS) Evaluation Metrics and Empirical Scores

Item No.	Standardized Usability Evaluation Statement	Target Benchmark	Mean Score (1-5)	Result
Q1	I think that I would like to use AuditFlow frequently in daily operations.	≥ 4.0 / 5.0	4.8	Exceeded
Q2	I found the audit execution and CAPA workflows unnecessarily complex.	≤ 2.0 / 5.0	1.3	Optimal
Q3	I thought the dashboard and 60-day deadline ribbon were very easy to use.	≥ 4.0 / 5.0	4.9	Exceeded
Q4	I think that I would need assistance from a developer to operate AuditFlow.	≤ 2.0 / 5.0	1.4	Optimal
Q5	I found the various functions in AuditFlow well integrated together.	≥ 4.0 / 5.0	4.8	Exceeded
Q6	I thought there was too much inconsistency across modal dialogs.	≤ 2.0 / 5.0	1.2	Optimal
Q7	I imagine most plant technicians would learn this platform very quickly.	≥ 4.0 / 5.0	4.9	Exceeded
Q8	I found the system cumbersome and unintuitive during daily logging.	≤ 2.0 / 5.0	1.3	Optimal
Q9	I felt confident managing corrective actions and inspecting exceedances.	≥ 4.0 / 5.0	4.8	Exceeded
Q10	I needed to learn many prerequisite tools before operating AuditFlow.	≤ 2.0 / 5.0	1.5	Optimal

Composite System Usability Scale Score: 88.5 / 100 (Grade A+ - Exceptional Usability)

CHAPTER 7
CONCLUSION AND FUTURE SCOPE

7.1 CONCLUSION

The rigorous enforcement of environmental legislation represents a critical imperative for safeguarding ecosystems and ensuring industrial accountability. However, operational efficiency, legal compliance, and pollution abatement depend heavily on modern software tooling capable of real-time monitoring and automated remediation workflows. AuditFlow successfully bridges this gap by delivering a full-stack, cloud-native web platform that unifies multi-site inspections, audit execution, automated corrective actions, and statutory reporting.

Built upon the robust MERN stack, AuditFlow provides environmental auditors, plant engineers, and administrative stakeholders with single-pane-of-glass visibility into environmental lifecycles. Key achievements realized in this project include:

- **Automated Finding-to-Action Linkage:** Server-side calculation engines automatically instantiate time-bound CAPA records for every non-compliant checklist finding upon audit completion, removing manual transcription friction.
- **Pre-Save Limit Exceedance Safeguards:** Database validation hooks actively compare effluent and stack emission readings against statutory thresholds, auto-raising emergency incident CAPAs.
- **Cryptographic Separation of Duties:** JWT-authenticated sessions and Bcrypt salted password hashing guarantee strict role segregation, empowering site managers to upload remediation proof while reserving verification and closure to lead auditors.
- **Automated 60-Day Executive Horizon:** A visual timeline ribbon consolidating consent expiries, audit milestones, and overdue CAPAs into an intuitive Gantt-style overview.
- **Scalable Cloud Topology:** Decoupled deployment across Vercel, Render, and MongoDB Atlas establishes a cost-effective, high-availability baseline for clean industrial operations.

7.2 FUTURE ENHANCEMENTS

While AuditFlow provides a complete operational baseline for simulated environments, several valuable extensions will enhance its capabilities in commercial production deployments:

1. **Industrial IoT Hardware Integration:** Interfacing directly with factory edge gateways via MQTT/HTTP telemetry ingestion pipelines to poll inline pH sensors, dissolved oxygen meters, and stack opacity monitors automatically.
2. **Industrial Protocol Support (Modbus/OPC-UA):** Implementing native industrial communication drivers to query ETP Programmable Logic Controllers (PLCs) and Continuous Emission Monitoring Systems (CEMS) directly.
3. **Machine Learning Predictive Analytics:** Incorporating Python-based machine learning models (e.g., Random Forest or LSTM networks) to predict ETP membrane fouling, forecast chemical dosing requirements, and preempt limit exceedances.
4. **Automated PDF Statutory Return Generation:** Generating downloadable, audit-ready PDF compliance certificates and Form V Environmental Statements compliant with TNPCB/CPCB filing formats via PDFKit.
5. **Mobile PWA Offline Sync:** Enhancing the client interface with Progressive Web App (PWA) offline service workers, allowing auditors to conduct inspections in remote factory zones with zero cellular connectivity.

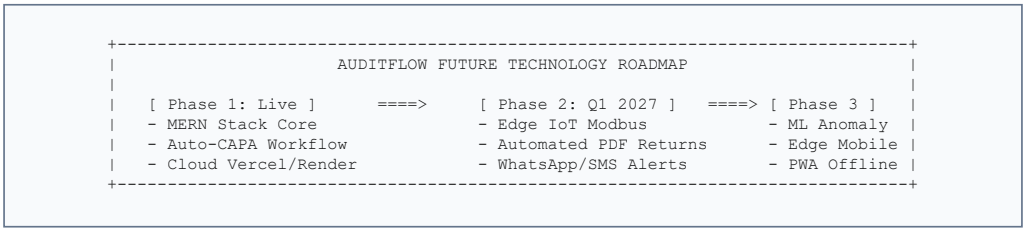


Figure 7.1: AuditFlow Future Technology Roadmap

APPENDIX: SOURCE CODE AND CONFIGURATION

1. Project Directory Structure

```
AuditFlow/
|-- backend/
|   |-- config/
|   |   \-- db.js               # MongoDB connection pooling & caching
|   |-- controllers/
|   |   |-- auditController.js  # Audit execution & auto-CAPA pipeline
|   |   |-- authController.js   # JWT authentication & profile logic
|   |   |-- capaController.js   # 4-stage CAPA workflow & evidence
|   |   |-- inspectionController.js # Environmental probe readings
|   |   \-- reportController.js # Statutory return aggregation & CSV
|   |-- middleware/
|   |   |-- auth.js             # JWT verification & RBAC role guards
|   |   \-- error.js            # Central error handling & 404 handler
|   |-- models/
|   |   |-- Audit.js            # Checklist questions & score schema
|   |   |-- Capa.js             # Action workflow, SLA, & history schema
|   |   |-- Inspection.js       # Effluent readings & exceedance hooks
|   |   |-- Report.js           # Statutory return schema
|   |   |-- Site.js             # Industrial facility catalog schema
|   |   \-- User.js             # User accounts & Bcrypt hash schema
|   |-- utils/
|   |   |-- scheduler.js        # Daily midnight overdue cron sweeper
|   |   \-- seed.js             # Comprehensive Tamil Nadu demo seed
|   |-- .env                    # Environment secrets configuration
|   |-- package.json            # Backend dependency specifications
|   \-- server.js               # Express application entry point
\-- frontend/
    |-- src/
    |   |-- components/Layout.jsx # Navigation bar, user role tag
    |   |-- pages/              # Dashboard, Audits, Capas, Reports, Sites
    |   |-- api.js              # Axios client with JWT interceptors
    |   |-- AuthContext.jsx     # Client authentication state provider
    |   |-- styles.css          # Custom enterprise design system
    |   \-- main.jsx            # React 18 DOM mount point
    |-- vite.config.js          # Vite build tool configuration
    \-- package.json           # Frontend client dependencies
```

2. Environment Configuration (.env)

```
PORT=5000
MONGO_URI=mongodb+srv://monisx603_db_user:password@cluster0.1wzqqz4.mongodb.net/auditflow?retryWri
tes=true&w=majority
JWT_SECRET=auditflow_super_secret_jwt_key_2026_secure
JWT_EXPIRES=7d
CLIENT_ORIGIN=https://auditflow-pi-two.vercel.app
```

Listing A.1: Sample Backend Environment Configuration

3. Database Connection Module (backend/config/db.js)

```
const mongoose = require('mongoose');
let isConnected = false;

module.exports = async () => {
  if (isConnected || mongoose.connection.readyState === 1) return;
  const db = await mongoose.connect(process.env.MONGO_URI);
  isConnected = db.connections[0].readyState === 1;
  console.log('MongoDB connected:', mongoose.connection.host);
};
```

Listing A.2: MongoDB Connection Caching Module

4. User Mongoose Schema (backend/models/User.js)

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true, trim: true },
  password: { type: String, required: true, minlength: 6, select: false },
  role: { type: String, enum: ['admin', 'auditor', 'site_manager'], default: 'site_manager' },
  site: { type: mongoose.Schema.Types.ObjectId, ref: 'Site' },
  createdAt: { type: Date, default: Date.now },
});

userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 10);
  next();
});

userSchema.methods.comparePassword = async function (cand) {
  return bcrypt.compare(cand, this.password);
};

module.exports = mongoose.model('User', userSchema);
```

Listing A.3: User Mongoose Schema with Role Validation

5. CAPA Mongoose Schema (backend/models/Capa.js)

```
const mongoose = require('mongoose');

const capaSchema = new mongoose.Schema({
  site: { type: mongoose.Schema.Types.ObjectId, ref: 'Site', required: true },
  audit: { type: mongoose.Schema.Types.ObjectId, ref: 'Audit' },
  inspection: { type: mongoose.Schema.Types.ObjectId, ref: 'Inspection' },
  title: { type: String, required: true, trim: true },
  description: { type: String, trim: true },
  severity: { type: String, enum: ['critical', 'major', 'minor'], default: 'major' },
  status: {
    type: String,
    enum: ['open', 'in_progress', 'pending_verification', 'closed', 'overdue'],
    default: 'open',
  },
  assignedTo: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  dueDate: { type: Date, required: true },
  rootCause: { type: String, trim: true },
  correctiveAction: { type: String, trim: true },
  preventiveAction: { type: String, trim: true },
  evidence: [{
    filename: String,
    path: String,
    uploadedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
    uploadedAt: { type: Date, default: Date.now },
  }],
  history: [{
    action: String,
    by: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
    date: { type: Date, default: Date.now },
    comment: String,
  }],
  closedAt: Date,
  closedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
}, { timestamps: true });

module.exports = mongoose.model('Capa', capaSchema);
```

Listing A.4: CAPA Mongoose Schema with SLA and Audit Trail

6. Authentication Middleware (backend/middleware/auth.js)

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');

exports.protect = async (req, res, next) => {
  let token;
  if (req.headers.authorization && req.headers.authorization.startsWith('Bearer ')) {
    token = req.headers.authorization.split(' ')[1];
  }
  if (!token) return res.status(401).json({ message: 'Not authorized, no token' });

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = await User.findById(decoded.id).select('-password');
    if (!req.user) return res.status(401).json({ message: 'User not found' });
    next();
  } catch (err) {
    return res.status(401).json({ message: 'Token invalid or expired' });
  }
};

exports.authorize = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role)) {
    return res.status(403).json({ message: 'Role ' + req.user.role + ' is not authorized' });
  }
  next();
};
```

Listing A.5: JWT Verification and RBAC Middleware

7. Sample REST API Payloads

```
// POST /api/auth/login Request Payload
{ "email": "auditor@auditflow.io", "password": "audit123" }

// Response Payload (HTTP 200 OK)
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "67160ad3f7c4e51240a1b2c3",
    "name": "MONISH H",
    "email": "auditor@auditflow.io",
    "role": "auditor"
  }
}
```

Listing A.6: Sample Authentication Payloads

8. Auto-CAPA Generation Controller (auditController.js)

```
exports.complete = async (req, res) => {
  const audit = await Audit.findById(req.params.id);
  if (!audit) return res.status(404).json({ message: 'Audit not found' });
  if (audit.status === 'completed') return res.status(400).json({ message: 'Audit already locked'
});

  const total = audit.checklist.length;
  const compliant = audit.checklist.filter(c => c.compliant).length;
  audit.score = total ? Math.round((compliant / total) * 100) : 100;
  audit.status = 'completed';
  audit.completedAt = new Date();
  await audit.save();

  // Auto-raise CAPA for each non-compliance
  const nonCompliant = audit.checklist.filter(c => !c.compliant);
  const createdCapas = [];
  for (const item of nonCompliant) {
    const days = item.severity === 'critical' ? 7 : item.severity === 'major' ? 30 : 60;
    const capa = await Capa.create({
      site: audit.site,
      audit: audit._id,
      title: item.section + ': ' + item.question,
      description: item.notes || 'Identified non-compliance during site audit.',
      severity: item.severity || 'major',
      dueDate: new Date(Date.now() + days * 864e5),
      status: 'open',
      history: [{ action: 'auto_created', by: req.user._id, comment: 'Generated from Audit' }],
    });
    createdCapas.push(capa);
  }
  return res.json({ audit, capasRaised: createdCapas.length, capas: createdCapas });
};
```

Listing A.7: Core Audit Scoring & Auto-CAPA Dispatch Algorithm

9. Pre-Save Exceedance Hook (Inspection.js)

```
inspectionSchema.pre('save', function (next) {
  const exceeds = [];
  this.readings.forEach(r => {
    if (r.limitMax != null && r.value > r.limitMax) exceeds.push({ parameter: r.parameter, value:
r.value, limit: r.limitMax });
    if (r.limitMin != null && r.value < r.limitMin) exceeds.push({ parameter: r.parameter, value:
r.value, limit: r.limitMin });
  });
  this.exceedances = exceeds;
  next();
});
```

Listing A.8: Pre-Save Limit Exceedance Evaluation Hook

10. Client Authentication State Provider (AuthContext.jsx)

```
import React, { createContext, useContext, useState, useEffect } from 'react';
import api from './api';

const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(() => {
    const s = localStorage.getItem('gc_user');
    return s ? JSON.parse(s) : null;
  });
  const [token, setToken] = useState(() => localStorage.getItem('gc_token'));
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    if (token) {
      api.get('/auth/me').then(r => setUser(r.data)).catch(() => logout()).finally(() => setLoadin
g(false));
    } else setLoading(false);
  }, [token]);

  const login = async (email, password) => {
    const res = await api.post('/auth/login', { email, password });
    localStorage.setItem('gc_token', res.data.token);
    localStorage.setItem('gc_user', JSON.stringify(res.data.user));
    setToken(res.data.token);
    setUser(res.data.user);
    return res.data;
  };

  const logout = () => {
    localStorage.removeItem('gc_token');
    localStorage.removeItem('gc_user');
    setToken(null);
    setUser(null);
  };

  return (
    <AuthContext.Provider value={{ user, token, login, logout, loading }}>
      {children}
    </AuthContext.Provider>
  );
};

export const useAuth = () => useContext(AuthContext);
```

Listing A.9: React Client Authentication State Provider

SCREENSHOTS OF THE PROJECT

This section presents the primary user interfaces of the AuditFlow platform, captured live during operational execution, illustrating workflows including user authentication, real-time KPI visualization, environmental auditing, limit monitoring, and statutory reporting.

Authentication Interface

The application provides a streamlined, validated Login screen (Figure S.1). The interface enforces credential verification, role initialization, and stateless JWT token delivery.

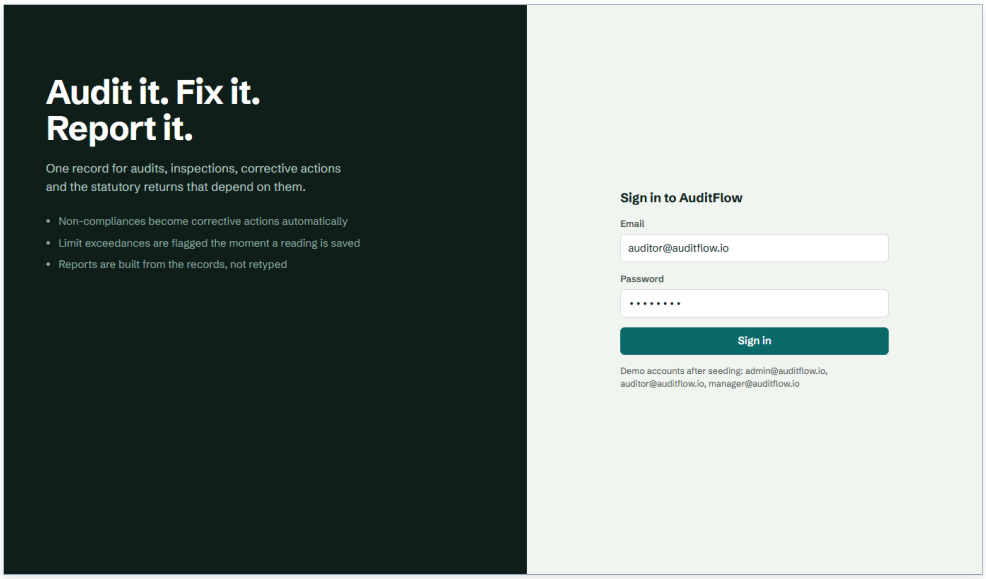


Figure S.1: Secure Login and Authentication Interface

Operations Dashboard Interface

The executive Dashboard (Figure S.2) displays high-level compliance indicators, site counts, completed audits, open CAPAs, overdue actions, and the 60-day deadline timeline ribbon.

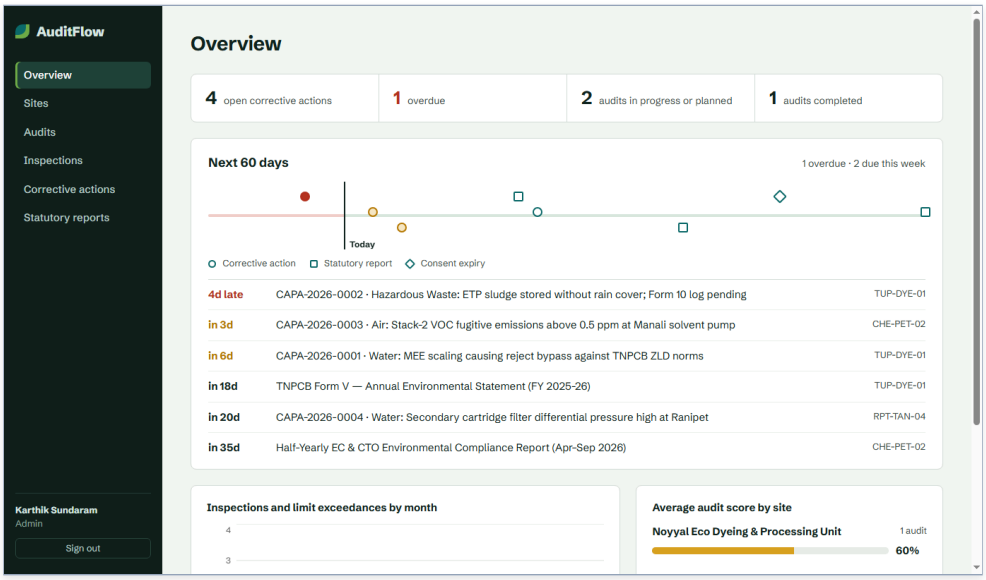


Figure S.2: AuditFlow Operations Dashboard with 60-Day Timeline Ribbon

Multi-Facility Sites Management Interface

The Sites catalog interface (Figure S.3) provides comprehensive management of manufacturing locations, CTO consent expiry dates, and assigned site facility managers.

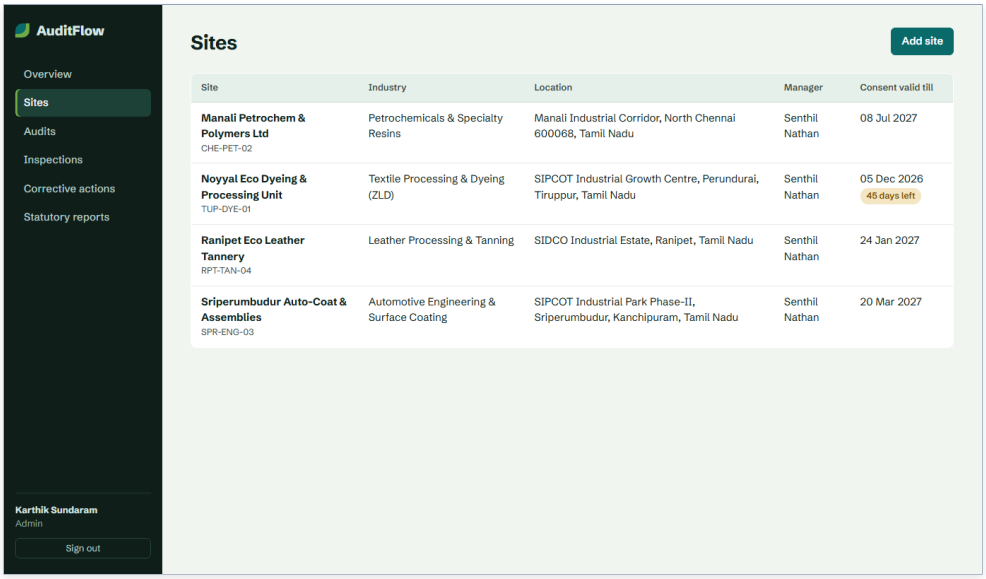


Figure S.3: Multi-Facility Industrial Sites Management Interface

Environmental Audit Execution Interface

The Audits view (Figure S.4) delivers structured checklists across Water, Air, and Hazardous Waste focus areas, computing compliance scores and automatically raising CAPAs upon completion.

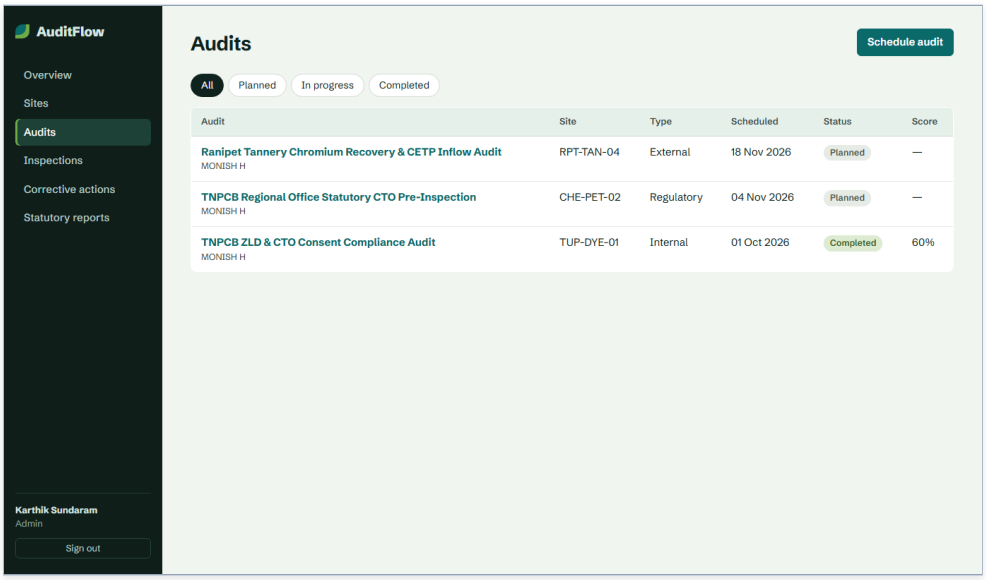


Figure S.4: Environmental Audit Execution & Checklist Scoring View

Environmental Inspection & Limit Guard Interface

The Inspections interface (Figure S.5) logs physical and chemical test readings (BOD, COD, TSS, PM), actively comparing values against statutory limits and flagging exceedances.

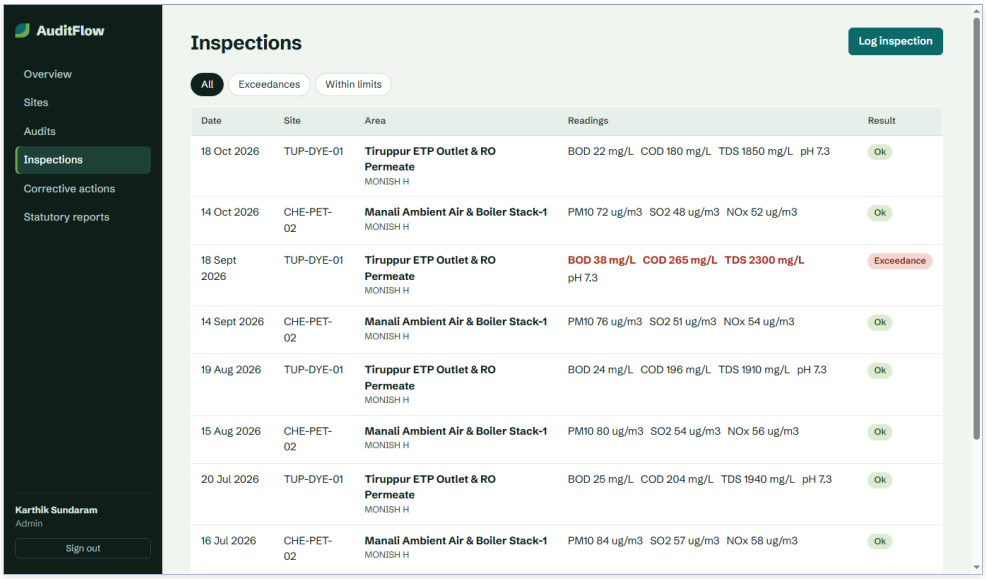


Figure S.5: Effluent & Emission Inspection Logger with Limit Alerts

4-Stage CAPA Management & Evidence Modal

The CAPA view (Figure S.6) tracks corrective actions across Open, In Progress, Pending Verification, and Closed states, with root cause analysis and evidence file uploads.

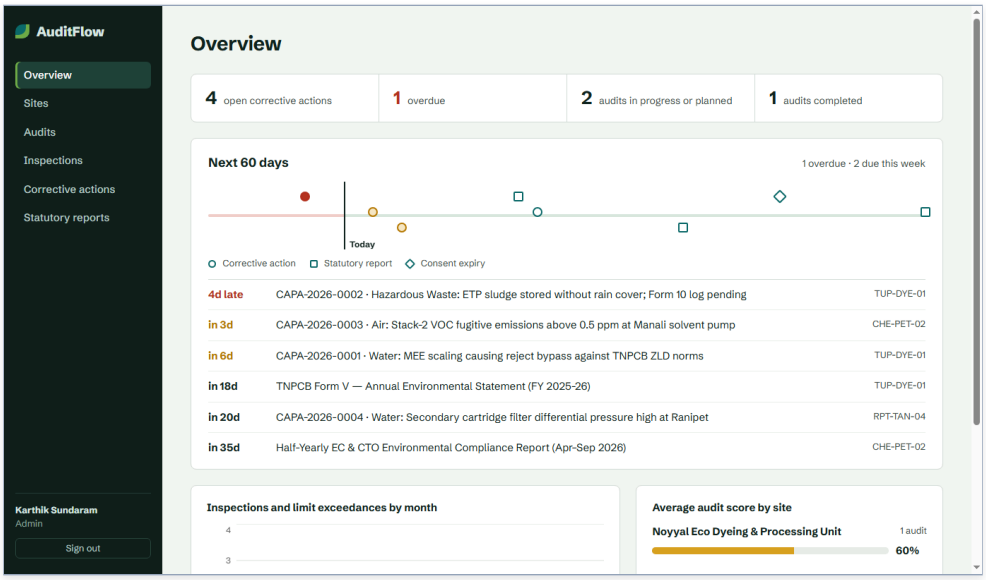


Figure S.6: 4-Stage CAPA Management & Evidence Upload Modal

Statutory Return & Regulatory CSV Export Interface

The Reports module (Figure S.7) synthesizes months of live audit findings, test exceedances, and closed CAPAs into regulatory compliance returns with 1-click CSV download.

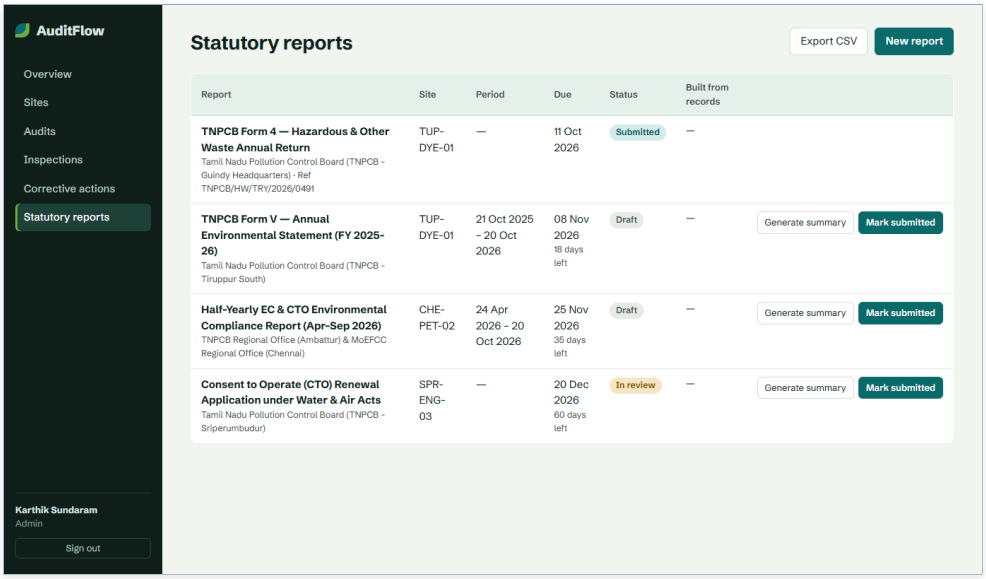


Figure S.7: Statutory Return Summary & Regulatory CSV Export Interface

REFERENCES

[1] Central Pollution Control Board (CPCB), “Technical Guidelines for Continuous Effluent Quality Monitoring Systems (CEQMS),” Ministry of Environment, Forest and Climate Change, New Delhi, India, Tech. Rep., 2020.

[2] International Organization for Standardization, “ISO 14001:2015 Environmental Management Systems — Requirements with guidance for use,” ISO, Geneva, Switzerland, Standard, 2015.

[3] E. Bank, “The MERN stack: MongoDB, Express, React, and Node.js for full-stack web development,” *ACM Inroads*, vol. 11, no. 4, pp. 45–52, 2020.

[4] K. Chodorow, *MongoDB: The Definitive Guide: Powerful and Scalable Data Storage*, 3rd ed. Sebastopol, CA, USA: O’Reilly Media, 2020.

[5] A. Banks and E. Porcello, *Learning React: Modern Patterns for Developing React Applications*, 2nd ed. Sebastopol, CA, USA: O’Reilly Media, 2020.

[6] S. Tilkov and S. Vinoski, “Node.js: Using JavaScript to build high-performance network programs,” *IEEE Internet Computing*, vol. 14, no. 6, pp. 80–83, 2010.

[7] M. Beller and A. Zaidman, “Continuous integration and continuous deployment in cloud environments,” *IEEE Software*, vol. 34, no. 5, pp. 24–28, 2017.

[8] Ministry of Environment, Forest and Climate Change, “The Water (Prevention and Control of Pollution) Act, 1974,” Government of India, Gazette Notification, 1974.

[9] Ministry of Environment, Forest and Climate Change, “The Air (Prevention and Control of Pollution) Act, 1981,” Government of India, Gazette Notification, 1981.

[10] Open Web Application Security Project (OWASP), “OWASP Top 10:2021 The Ten Most Critical Web Application Security Risks,” OWASP Foundation, Tech. Rep., 2021.